Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Гречко Е.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 30.10.26

Москва, 2026

Постановка задачи

Вариант 6.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы.

Также необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Задание варианта: произвести перемножение двух матриц, содержащих комплексные числа. Реализовать последовательную и параллельную версии алгоритма и измерить время их выполнения для разного количества потоков.

Общий метод и алгоритм решения

Использованные системные вызовы:

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg); – создаёт новый поток, выполняющий функцию start_routine с аргументом arg.
- int pthread_join(pthread_t thread, void **retval); – ожидает завершения потока и освобождает его ресурсы.
- int clock_gettime(clockid_t clockid, struct timespec *tp); – получает текущее время (CLOCK_MONOTONIC используется для замеров).
- ssize_t write(int fd, const void *buf, size_t count); – выводит результаты и сообщения об ошибках.
- pid_t getpid(void); – возвращает идентификатор процесса (нужен для поиска процесса в ps).
- void exit(int status); – завершает процесс при ошибке.

Программа написана на языке Си и запускается в ОС Linux (WSL2, Ubuntu). Для работы с потоками используется библиотека POSIX Threads, вывод выполняется системным вызовом write без использования stdio.h. Программа состоит из файлов matrix.h и matrix.c (комплексные числа и операции с матрицами), logger.h и logger.c (вывод строк и чисел через write, сообщения об ошибках) и main.c (разбор ключей запуска, последовательная и параллельная версии умножения, замер времени и вывод результатов).

Представление данных. Комплексное число хранится в структуре Complex из двух полей: действительной и мнимой части. Матрица (структура Matrix) хранит число строк, число столбцов и одномерный массив элементов по строкам: элемент (i, j) находится в ячейке data[i·cols + j]. Произведение комплексных чисел вычисляется по формуле $(a + bi)(c + di) = (ac − bd) + (ad + bc)i$.

Алгоритм умножения. Элемент результата $C = A \cdot B$ равен $c_{ij} = \Sigma\, a_{ik} \cdot b_{kj}$. Циклы идут в порядке i–k–j: для каждого элемента a_{ik} к строке i матрицы C прибавляется строка k матрицы B, умноженная на a_{ik}. При таком порядке строки B и C читаются и записываются последовательно, что эффективнее для кэша; нулевые элементы a_{ik} пропускаются. Сложность алгоритма – $O(N^3)$ для матриц N×N.

Параллельная версия. Строки матрицы A (а значит, и строки результата C) делятся на p непрерывных блоков, где p – значение ключа -p: каждый поток получает N / p строк, а первые N mod p потоков – на одну строку больше. Главный поток создаёт p потоков функцией pthread_create,

передавая каждому структуру ThreadArgs с указателями на матрицы и границами своего блока строк (row_start, row_end). Каждый поток вычисляет свои строки C функцией matrix_multiply_thread. Последовательная версия вызывает ту же функцию для всех строк в одном потоке, поэтому обе версии выполняют одинаковые вычисления.

Синхронизация. Матрицы A и B потоки только читают, а в матрицу C каждый поток записывает только свои строки, поэтому общих изменяемых данных нет и мьютексы не требуются. Главный поток ожидает завершения всех потоков функцией pthread_join и только после этого фиксирует время и использует результат.

Ключи запуска: -n – размер матриц N×N (по умолчанию 256); -p – количество потоков (обязательный); -s – начальное значение генератора случайных чисел (по умолчанию 42); -r min:max – диапазон значений действительной и мнимой частей (по умолчанию 1:10); -h – справка. Матрицы заполняются случайными комплексными числами функцией matrix_fill_random.

Замер времени. Время измеряется функцией clock_gettime(CLOCK_MONOTONIC) для последовательной и параллельной версий на одних и тех же матрицах; в время параллельной версии входят создание потоков и ожидание их завершения. Для оценки параллельного алгоритма используются две метрики:

- **Ускорение** $S_p = T_1 / T_p$ – показывает, во сколько раз параллельная программа выполняется быстрее последовательной, где T_1 – время выполнения последовательного алгоритма, T_p – время выполнения параллельного алгоритма на p потоках.
- **Эффективность** $E_p = S_p / p$ – показывает, какая доля вычислительных ресурсов потоков используется полезно, где p – количество потоков; идеальное значение равно 1 (линейное ускорение).

Код программы

main.c

```
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <pthread.h>
#include <getopt.h>
#include <sys/types.h>

#include "matrix.h"
#include "logger.h"

typedef struct {
    int matrix_size;
    int n_threads;
    int random_seed;
    int range_min;
    int range_max;
} ProgramArgs;

typedef struct {
    const Matrix *a;
    const Matrix *b;
    Matrix *c;
    int row_start;
    int row_end;
} ThreadArgs;

static void *matrix_multiply_thread(void *_args) {
    ThreadArgs *args = (ThreadArgs *)_args;
    const Matrix *a = args->a;
    const Matrix *b = args->b;
```

```
    Matrix *c = args->c;

    int a_cols = a->cols;
    int b_cols = b->cols;

    for (int i = args->row_start; i < args->row_end; i++) {
        for (int k = 0; k < a_cols; k++) {
            Complex a_ik = a->data[i * a_cols + k];
            if (a_ik.real == 0.0 && a_ik.imag == 0.0)
                continue;
            for (int j = 0; j < b_cols; j++) {
                Complex b_kj = b->data[k * b_cols + j];
                Complex product = complex_multiply(a_ik, b_kj);
                int idx = i * b_cols + j;
                c->data[idx] = complex_add(c->data[idx], product);
            }
        }
    }
    return NULL;
}

typedef struct {
    long sec;
    long nsec;
} TimePoint;

static TimePoint get_time(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (TimePoint){ .sec = ts.tv_sec, .nsec = ts.tv_nsec };
}

static double time_diff_ms(TimePoint start, TimePoint end) {
    double sec = (double)(end.sec - start.sec);
    double nsec = (double)(end.nsec - start.nsec);
    return (sec * 1000.0) + (nsec / 1000000.0);
}

static double benchmark_sequential(const Matrix *a, const Matrix *b, Matrix *c) {
    matrix_clear(c);
    ThreadArgs args = { .a = a, .b = b, .c = c, .row_start = 0, .row_end = a->rows };
    TimePoint t_start = get_time();
    matrix_multiply_thread(&args);
    TimePoint t_end = get_time();
    return time_diff_ms(t_start, t_end);
}

static double benchmark_parallel(const Matrix *a, const Matrix *b, Matrix *c, int n_threads) {
    matrix_clear(c);

    pthread_t *threads = malloc(n_threads * sizeof(pthread_t));
    ThreadArgs *thread_args = malloc(n_threads * sizeof(ThreadArgs));

    int total_rows = a->rows;
    int rows_per_thread = total_rows / n_threads;
    int remainder = total_rows % n_threads;

    TimePoint t_start = get_time();

    int current_row = 0;
    for (int t = 0; t < n_threads; t++) {
        int rows_for_this = rows_per_thread + (t < remainder ? 1 : 0);
        thread_args[t] = (ThreadArgs){
            .a = a, .b = b, .c = c,
            .row_start = current_row,
            .row_end = current_row + rows_for_this,
        };
        current_row += rows_for_this;

        if (pthread_create(&threads[t], NULL, matrix_multiply_thread,
                           &thread_args[t]) != 0) {
            free(thread_args);
            free(threads);
```

```
            log_error("не получилось создать поток");
        }
    }


    for (int t = 0; t < n_threads; t++)
        pthread_join(threads[t], NULL);
    TimePoint t_end = get_time();

    free(thread_args);
    free(threads);

    return time_diff_ms(t_start, t_end);
}

static void print_usage(const char *prog_name) {
    log_write_str(STDOUT_FILENO, "Как пользоваться: ");
    log_write_str(STDOUT_FILENO, prog_name);
    log_write_str(STDOUT_FILENO,
        " [по флагам-опциям]\n"
        "Опции:\n"
        "  -n <size>    размер матрцы N x N (default: 256)\n"
        "  -p <threads> количество потоков\n"
        "  -s <seed>    seed случайной генерации (default: 42)\n"
        "  -r <min:max> разброс значений (default: 1:10)\n"
        "  -h           подсказка \n");
}

static ProgramArgs parse_args(int argc, char **argv) {
    ProgramArgs args = {
        .matrix_size = 256,
        .n_threads = 0,
        .random_seed = 42,
        .range_min = 1,
        .range_max = 10
    };

    int opt;
    while ((opt = getopt(argc, argv, "n:p:s:r:h")) != -1) {
        switch (opt) {
        case 'n':
            args.matrix_size = atoi(optarg);
            break;
        case 'p':
            args.n_threads = atoi(optarg);
            break;
        case 's':
            args.random_seed = atoi(optarg);
            break;
        case 'r': {
            char *colon = strchr(optarg, ':');
            if (colon) {
                *colon = '\0';
                args.range_min = atoi(optarg);
                args.range_max = atoi(colon + 1);
            }
            break;
        }
        case 'h':
            print_usage(argv[0]);
            exit(EXIT_SUCCESS);
        default:
            print_usage(argv[0]);
            exit(EXIT_FAILURE);
        }
    }

    if (args.matrix_size <= 0)
        log_error("размер матрицы желательно больше нуля делать");
    if (args.n_threads <= 0)
        log_error("необходимо указать количество потоков");

    return args;
```

```
}

int main(int argc, char **argv)
{
    ProgramArgs args = parse_args(argc, argv);

    pid_t pid = getpid();
    log_write_str(STDOUT_FILENO, "PID процесса: ");
    log_write_int(STDOUT_FILENO, pid);
    log_write_str(STDOUT_FILENO, "\n\n");


    srand(args.random_seed);

    Matrix *a = matrix_create(args.matrix_size, args.matrix_size);
    Matrix *b = matrix_create(args.matrix_size, args.matrix_size);
    Matrix *c_seq = matrix_create(args.matrix_size, args.matrix_size);
    Matrix *c_par = matrix_create(args.matrix_size, args.matrix_size);

    if (!a || !b || !c_seq || !c_par)
        log_error("не удалось создать матрицу");

    matrix_fill_random(a, args.range_min, args.range_max);
    matrix_fill_random(b, args.range_min, args.range_max);

    double t_seq = benchmark_sequential(a, b, c_seq);

    double t_par = benchmark_parallel(a, b, c_par, args.n_threads);

    double speedup = (t_par > 0.0) ? t_seq / t_par : 0.0;
    double efficiency = (args.n_threads > 0) ? speedup / args.n_threads : 0.0;

    log_write_str(STDOUT_FILENO,
        "-------------------------------------------------------------------------------\n"
        "| Threads | Seq (ms)    | Par (ms)    | Speedup     | Efficiency  |\n"
        "-------------------------------------------------------------------------------\n"
        "| ");
    log_write_int(STDOUT_FILENO, args.n_threads);
    log_write_str(STDOUT_FILENO, "        | ");
    log_write_double(STDOUT_FILENO, t_seq, 3);
    log_write_str(STDOUT_FILENO, "    | ");
    log_write_double(STDOUT_FILENO, t_par, 3);
    log_write_str(STDOUT_FILENO, "    | ");
    log_write_double(STDOUT_FILENO, speedup, 3);
    log_write_str(STDOUT_FILENO, "    | ");
    log_write_double(STDOUT_FILENO, efficiency, 3);
    log_write_str(STDOUT_FILENO, "    |\n"
        "-------------------------------------------------------------------------------\n\n");


    matrix_destroy(&a);
    matrix_destroy(&b);
    matrix_destroy(&c_seq);
    matrix_destroy(&c_par);

    return 0;
}
```

matrix.h

```
#ifndef MATRIX_H
#define MATRIX_H

#include <stdlib.h>
#include <stdbool.h>
#include <unistd.h>

typedef struct {
    double real;
    double imag;
} Complex;

Complex complex_create(double real, double imag);
```

```
Complex complex_add(Complex a, Complex b);
Complex complex_multiply(Complex a, Complex b);

typedef struct {
    int rows;
    int cols;
    Complex *data;
} Matrix;

Matrix* matrix_create(int rows, int cols);
void matrix_destroy(Matrix **m);
void matrix_clear(Matrix *m);


bool matrix_valid_index(const Matrix *m, int row, int col);

void matrix_fill_random(Matrix *m, int min_val, int max_val);
void matrix_fill_sequential(Matrix *m);
bool matrix_copy(const Matrix *src, Matrix *dst);
bool matrix_equal(const Matrix *a, const Matrix *b);

Matrix* matrix_multiply(const Matrix *a, const Matrix *b);

void matrix_print(const Matrix *m, int fd);
Matrix* matrix_read_from_fd(int fd);

int matrix_size(const Matrix *m);
Matrix* matrix_transpose(const Matrix *m);

#endif
```

matrix.c

```
#include "matrix.h"
#include "logger.h"

#include <stdlib.h>
#include <time.h>
#include <fcntl.h>


Complex complex_create(double real, double imag)
{
    Complex c;
    c.real = real;
    c.imag = imag;
    return c;
}

Complex complex_add(Complex a, Complex b)
{
    return complex_create(a.real + b.real, a.imag + b.imag);
}

Complex complex_multiply(Complex a, Complex b)
{
    double real = a.real * b.real - a.imag * b.imag;
    double imag = a.real * b.imag + a.imag * b.real;
    return complex_create(real, imag);
}


Matrix* matrix_create(int rows, int cols)
{
    if (rows <= 0 || cols <= 0)
        return NULL;

    Matrix *m = (Matrix*)malloc(sizeof(Matrix));
    if (!m) return NULL;

    m->rows = rows;
    m->cols = cols;
    m->data = (Complex*)calloc(rows * cols, sizeof(Complex));
```

```
    if (!m->data) {
        free(m);
        return NULL;
    }

    return m;
}

void matrix_destroy(Matrix **m)
{
    if (m && *m) {
        free((*m)->data);
        free(*m);
        *m = NULL;
    }
}

void matrix_clear(Matrix *m)
{
    if (m && m->data) {
        for (int i = 0; i < m->rows * m->cols; i++) {
            m->data[i] = complex_create(0.0, 0.0);
        }
    }
}


bool matrix_valid_index(const Matrix *m, int row, int col)
{
    return (m != NULL && row >= 0 && row < m->rows &&
            col >= 0 && col < m->cols);
}

void matrix_fill_random(Matrix *m, int min_val, int max_val)
{
    if (!m || !m->data) return;

    for (int i = 0; i < m->rows * m->cols; i++) {
        double real = min_val + rand() % (max_val - min_val + 1);
        double imag = min_val + rand() % (max_val - min_val + 1);
        m->data[i] = complex_create(real, imag);
    }
}

void matrix_fill_sequential(Matrix *m)
{
    if (!m || !m->data) return;

    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
            double real = (i + 1) * 10 + (j + 1);
            double imag = (i + 1) + (j + 1);
            m->data[i * m->cols + j] = complex_create(real, imag);
        }
    }
}

bool matrix_copy(const Matrix *src, Matrix *dst)
{
    if (!src || !dst || src->rows != dst->rows || src->cols != dst->cols)
        return false;

    for (int i = 0; i < src->rows * src->cols; i++)
        dst->data[i] = src->data[i];

    return true;
}



Matrix* matrix_multiply(const Matrix *a, const Matrix *b)
{
```

```
    if (!a || !b || a->cols != b->rows) {
        log_error("incompatible matrix dimensions for multiplication");
    }

    Matrix *c = matrix_create(a->rows, b->cols);
    if (!c) return NULL;

    int a_rows = a->rows;
    int a_cols = a->cols;
    int b_cols = b->cols;

    for (int i = 0; i < a_rows; i++) {
        for (int k = 0; k < a_cols; k++) {
            Complex a_ik = a->data[i * a_cols + k];

            if (a_ik.real == 0.0 && a_ik.imag == 0.0)
                continue;

            for (int j = 0; j < b_cols; j++) {
                Complex b_kj = b->data[k * b_cols + j];
                Complex product = complex_multiply(a_ik, b_kj);
                int idx = i * b_cols + j;
                c->data[idx] = complex_add(c->data[idx], product);
            }
        }
    }

    return c;
}

static bool read_char(int fd, char *c)
{
    ssize_t n = read(fd, c, 1);
    return (n == 1);
}

static void skip_whitespace(int fd)
{
    char c;
    while (read_char(fd, &c)) {
        if (c != ' ' && c != '\t' && c != '\n' && c != '\r') {
            lseek(fd, -1, SEEK_CUR);
            break;
        }
    }
}

static bool read_int_fd(int fd, int *value)
{
    skip_whitespace(fd);

    char c;
    if (!read_char(fd, &c))
        return false;

    int sign = 1;
    if (c == '-') {
        sign = -1;
        if (!read_char(fd, &c))
            return false;
    } else if (c == '+') {
        if (!read_char(fd, &c))
            return false;
    }

    if (c < '0' || c > '9')
        return false;

    int result = c - '0';
    while (read_char(fd, &c)) {
        if (c < '0' || c > '9') {
            lseek(fd, -1, SEEK_CUR);
            break;
```

```
        }
        result = result * 10 + (c - '0');
    }

    *value = sign * result;
    return true;
}

static bool read_double_fd(int fd, double *value)
{
    skip_whitespace(fd);

    char c;
    if (!read_char(fd, &c))
        return false;

    int sign = 1;
    if (c == '-') {
        sign = -1;
        if (!read_char(fd, &c))
            return false;
    } else if (c == '+') {
        if (!read_char(fd, &c))
            return false;
    }

    if (c < '0' || c > '9')
        return false;

    double result = c - '0';
    while (read_char(fd, &c)) {
        if (c < '0' || c > '9') {
            if (c == '.') break;
            lseek(fd, -1, SEEK_CUR);
            break;
        }
        result = result * 10 + (c - '0');
    }

    if (c == '.') {
        double frac = 0.1;
        while (read_char(fd, &c)) {
            if (c < '0' || c > '9') {
                lseek(fd, -1, SEEK_CUR);
                break;
            }
            result += (c - '0') * frac;
            frac *= 0.1;
        }
    }

    *value = sign * result;
    return true;
}

Matrix* matrix_read_from_fd(int fd)
{
    int rows, cols;
    if (!read_int_fd(fd, &rows) || !read_int_fd(fd, &cols))
        return NULL;

    Matrix *m = matrix_create(rows, cols);
    if (!m) return NULL;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            double real, imag;
            if (!read_double_fd(fd, &real) || !read_double_fd(fd, &imag)) {
                matrix_destroy(&m);
                return NULL;
            }
            m->data[i * cols + j] = complex_create(real, imag);
        }
```

```
    }

    return m;
}

void matrix_print(const Matrix *m, int fd)
{
    if (!m || !m->data) {
        log_write_str(fd, "(empty matrix)\n");
        return;
    }

    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
            Complex c = m->data[i * m->cols + j];
            log_write_double(fd, c.real, 1);
            if (c.imag >= 0)
                log_write_str(fd, "+");
            log_write_double(fd, c.imag, 1);
            log_write_str(fd, "i");
            if (j < m->cols - 1)
                log_write_str(fd, "  ");
        }
        log_write_str(fd, "\n");
    }
}

int matrix_size(const Matrix *m)
{
    return m ? m->rows * m->cols : 0;
}

Matrix* matrix_transpose(const Matrix *m)
{
    if (!m) return NULL;

    Matrix *t = matrix_create(m->cols, m->rows);
    if (!t) return NULL;

    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
            t->data[j * m->rows + i] = m->data[i * m->cols + j];
        }
    }

    return t;
}
```

logger.h

```
#ifndef LOGGER_H
#define LOGGER_H

#include <stddef.h>
#include <unistd.h>


typedef enum {
    LOG_LEVEL_ERROR
} LogLevel;


void log_error(const char *msg);

ssize_t log_write(int fd, const char *msg, size_t len);
ssize_t log_write_str(int fd, const char *msg);
void log_write_int(int fd, int value);
void log_write_double(int fd, double value, int precision);

void log_printf(LogLevel level, const char *prefix, const char *msg);

#endif
```

logger.c

```
#include "logger.h"

#include <stdlib.h>
#include <string.h>

#define LOG_BUFFER_SIZE 256


ssize_t log_write(int fd, const char *msg, size_t len) {
    if (!msg || len == 0)
        return 0;

    ssize_t total = 0;
    while ((size_t)total < len) {
        ssize_t written = write(fd, msg + total, len - total);
        if (written < 0)
            return -1;
        total += written;
    }
    return total;
}

ssize_t log_write_str(int fd, const char *msg) {
    if (!msg)
        return 0;
    return log_write(fd, msg, strlen(msg));
}


void log_write_int(int fd, int value)
{
    char buf[12];
    int pos = sizeof(buf) - 1;
    buf[pos] = '\0';

    unsigned int uval;
    if (value < 0) {
        uval = (unsigned int)(-(value + 1)) + 1;
    } else {
        uval = (unsigned int)value;
    }

    do {
        buf[--pos] = '0' + (uval % 10);
        uval /= 10;
    } while (uval > 0);

    if (value < 0)
        buf[--pos] = '-';

    log_write(fd, buf + pos, strlen(buf + pos));
}

void log_write_double(int fd, double value, int precision)
{
    if (value < 0) {
        log_write_str(fd, "-");
        value = -value;
    }

    long long int_part = (long long)value;
    log_write_int(fd, (int)int_part);

    if (precision <= 0)
        return;

    log_write_str(fd, ".");

    double frac = value - (double)int_part;
    for (int i = 0; i < precision; i++) {
        frac *= 10;
```

```
        int digit = (int)frac;
        if (digit < 0) digit = 0;
        if (digit > 9) digit = 9;
        char c = '0' + digit;
        log_write(fd, &c, 1);
        frac -= digit;
    }
}

void log_error(const char *msg)
{
    if (!msg)
        return;
    log_write_str(STDERR_FILENO, "error: ");
    log_write_str(STDERR_FILENO, msg);
    size_t len = strlen(msg);

    if (len == 0 || msg[len - 1] != '\n')
        log_write_str(STDERR_FILENO, "\n");
    exit(EXIT_FAILURE);
}

void log_printf(LogLevel level, const char *prefix, const char *msg)
{
    int fd = level == LOG_LEVEL_ERROR ? STDERR_FILENO : STDOUT_FILENO;

    if (prefix)
        log_write_str(fd, prefix);
    if (msg)
        log_write_str(fd, msg);
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(OS_lab2 C)

set(CMAKE_C_STANDARD 11)

find_package(Threads REQUIRED)

add_executable(lab2 main.c matrix.c logger.c)
target_link_libraries(lab2 Threads::Threads)
target_compile_options(lab2 PRIVATE -O2)
```

Протокол работы программы

Сборка программы (WSL2, Ubuntu):

```
$ cmake -S . -B build
-- The C compiler identification is GNU 15.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Configuring done (0.8s)
-- Generating done (0.0s)
-- Build files have been written to: /tmp/lab2/build
$ cmake --build build
[ 25%] Building C object CMakeFiles/lab2.dir/main.c.o
[ 50%] Building C object CMakeFiles/lab2.dir/matrix.c.o
[ 75%] Building C object CMakeFiles/lab2.dir/logger.c.o
[100%] Linking C executable lab2
[100%] Built target lab2
```

Количество логических ядер системы:

```
$ nproc
```

8

Справка по ключам запуска:

```
$ ./build/lab2 -h
Как пользоваться: ./build/lab2 [по флагам-опциям]
Опции:
  -n <size>    размер матрцы N x N (default: 256)
  -p <threads> количество потоков
  -s <seed>    seed случайной генерации (default: 42)
  -r <min:max> разброс значений (default: 1:10)
  -h           подсказка
```

Запуск без обязательного ключа -p:

```
$ ./build/lab2 -n 100
error: необходимо указать количество потоков
```

Умножение матриц 1000×1000 на 8 потоках:

```
$ ./build/lab2 -n 1000 -p 8
PID процесса: 482

-------------------------------------------------------------------------------
| Threads | Seq (ms)    | Par (ms)    | Speedup     | Efficiency  |
-------------------------------------------------------------------------------
| 8        | 2591.393    | 472.547    | 5.483    | 0.685    |
-------------------------------------------------------------------------------
```

Демонстрация количества потоков средствами ОС:

```
$ ./build/lab2 -n 1500 -p 6 &
[1] 491
PID процесса: 491

$ ps -o pid,nlwp,cmd -p $!
    PID NLWP CMD
    491    7 ./build/lab2 -n 1500 -p 6
$ ps -T -p $!
    PID    SPID TTY          TIME CMD
    491     491 pts/0    00:00:07 lab2
    491     783 pts/0    00:00:00 lab2
    491     784 pts/0    00:00:00 lab2
    491     785 pts/0    00:00:00 lab2
    491     786 pts/0    00:00:00 lab2
    491     787 pts/0    00:00:00 lab2
    491     788 pts/0    00:00:00 lab2
-------------------------------------------------------------------------------
| Threads | Seq (ms)    | Par (ms)    | Speedup     | Efficiency  |
-------------------------------------------------------------------------------
| 6        | 7435.470    | 1335.241    | 5.568    | 0.928    |
-------------------------------------------------------------------------------

[1]+  Done                    ./build/lab2 -n 1500 -p 6
```

Команды ps выполнены во время параллельного умножения, после завершения последовательного (около 7 секунд, это видно по столбцу TIME главного потока). Утилита показывает NLWP = 7: шесть рабочих потоков, созданных pthread_create по значению ключа -p 6, и главный поток, который в это время ожидает их в pthread_join и вычислений не выполняет.

Замеры для разного количества потоков (N = 1000):

```
$ for p in 1 2 3 4 6 8 16 128 1024; do ./build/lab2 -n 1000 -p $p | grep "^| [0-9]"; done
| 1        | 1930.268    | 1928.409    | 1.000    | 1.000    |
| 2        | 1936.682    | 979.027    | 1.978    | 0.989    |
| 3        | 2022.298    | 686.253    | 2.946    | 0.982    |
| 4        | 2052.555    | 531.985    | 3.858    | 0.964    |
| 6        | 2023.793    | 452.821    | 4.469    | 0.744    |
| 8        | 2105.953    | 389.935    | 5.400    | 0.675    |
| 16        | 2143.623    | 446.259    | 4.803    | 0.300    |
| 128        | 2164.357    | 395.986    | 5.465    | 0.042    |
| 1024        | 2108.910    | 442.657    | 4.764    | 0.004    |
```

Результаты исследования

Замеры выполнены на процессоре Intel Core Ultra 7 258V (8 логических ядер: 4 производительных и 4 энергоэффективных ядра, без Hyper-Threading) в WSL2 (Ubuntu, ядро Linux 6.18), компилятор gcc 15.2 с оптимизацией -O2. Для каждого количества потоков выполнялся один запуск программы на одних и тех же матрицах (seed 42); время последовательной версии T_1 измеряется в каждом запуске. Таблица 1 составлена по приведённому выше протоколу: в ней 5 экспериментов с количеством потоков меньше количества логических ядер, 1 эксперимент с количеством потоков, равным количеству ядер, и 3 эксперимента с количеством потоков, значительно превышающим количество ядер.

Таблица 1 – Зависимость времени, ускорения и эффективности от количества потоков (N = 1000, среднее T_1 = 2054,3 мс)

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	1928,4	1,000	1,000
2	979,0	1,978	0,989
3	686,3	2,946	0,982
4	532,0	3,858	0,964
6	452,8	4,469	0,744
8	**389,9**	**5,400**	**0,675**
16	446,3	4,803	0,300
128	396,0	5,465	0,042
1024	442,7	4,764	0,004

Таблица 2 – Зависимость ускорения от размера матриц (среднее T_1 = 28,7 мс при N = 250, 245,0 мс при N = 500, 2054,3 мс при N = 1000, 7132,0 мс при N = 1500)

Число потоков	N = 250	N = 500	N = 1000	N = 1500
1	1,026	0,987	1,000	1,027
2	1,758	1,965	1,978	2,069
4	3,810	3,139	3,858	4,211
8	2,750	4,107	5,400	6,019
16	3,220	2,672	4,803	5,620
128	2,783	4,117	5,465	5,459
1024	0,680	2,991	4,764	5,815

Таблица 3 – Зависимость эффективности от размера матриц

Число потоков	N = 250	N = 500	N = 1000	N = 1500
1	1,026	0,987	1,000	1,027
2	0,879	0,982	0,989	1,034
4	0,952	0,784	0,964	1,052
8	0,343	0,513	0,675	0,752
16	0,201	0,167	0,300	0,351
128	0,021	0,032	0,042	0,042
1024	0,001	0,003	0,004	0,005

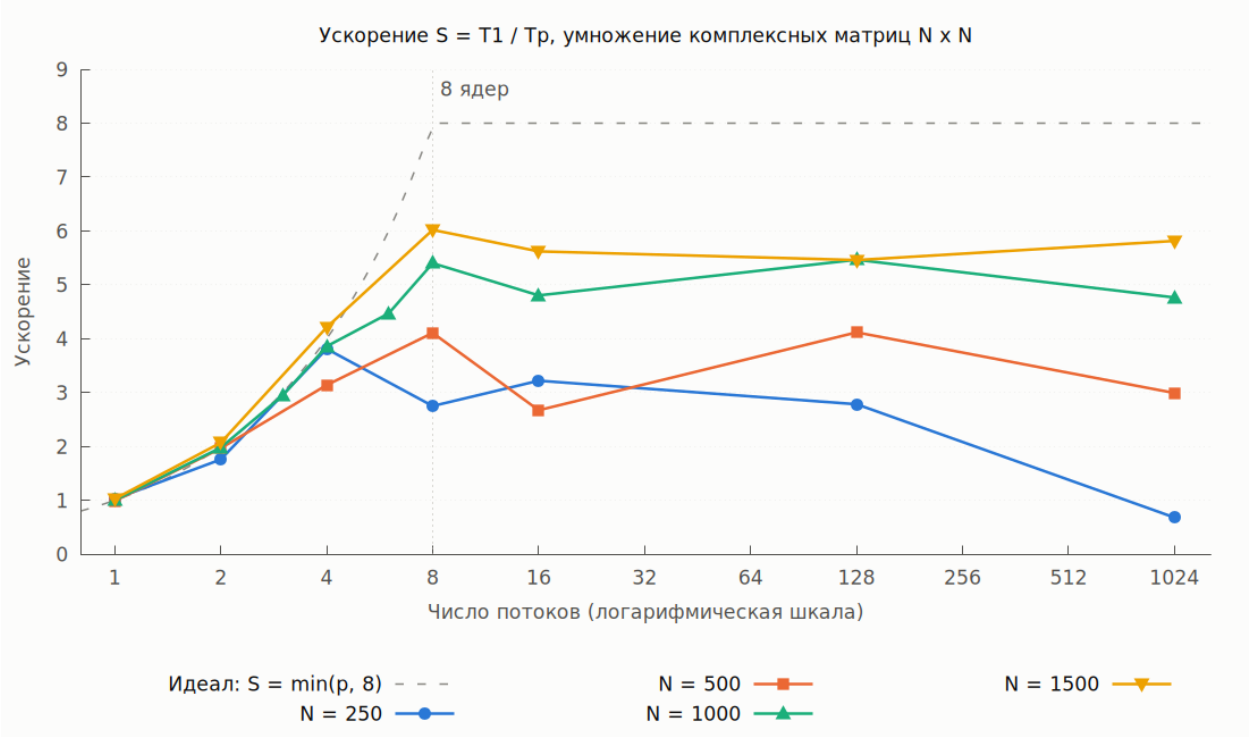


Рисунок 1 – Ускорение в зависимости от количества потоков

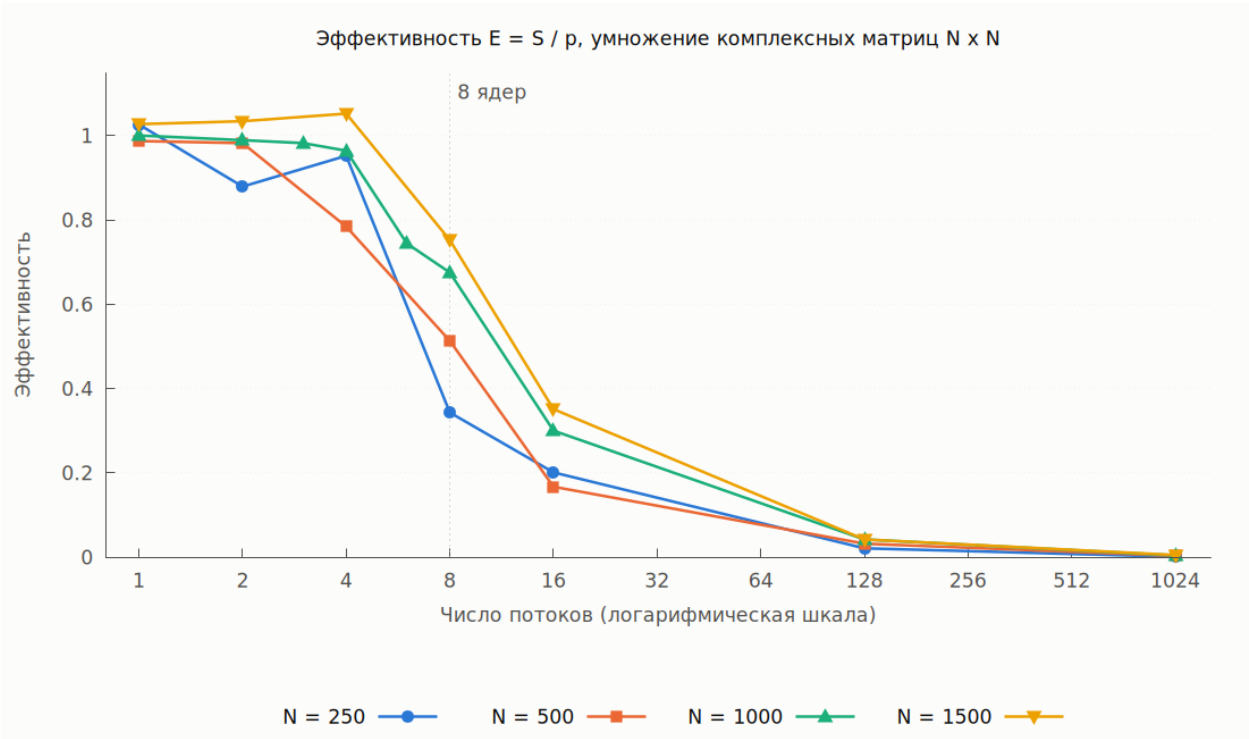


Рисунок 2 – Эффективность в зависимости от количества потоков

Вывод

В ходе лабораторной работы я реализовал последовательную и параллельную версии умножения матриц из комплексных чисел на языке Си с использованием POSIX Threads, научился создавать потоки, распределять между ними работу, ожидать их завершения и определять количество потоков процесса средствами ОС (ps).

При количестве потоков меньше количества логических ядер ускорение близко к линейному: на 2, 3 и 4 потоках эффективность составляет 0,99, 0,98 и 0,96, так как каждый поток вычисляет свои строки результата независимо от других и синхронизация между ними не нужна. На 6 и 8 потоках рост замедляется (на 8 потоках $S = 5{,}40$, $E = 0{,}68$): половина ядер процессора – более медленные

энергоэффективные ядра, а главный поток ждёт самый медленный поток; кроме того, каждый поток читает всю матрицу B (16 МБ при N = 1000), которая не помещается в кэш, и потоки делят общую пропускную способность памяти.

При количестве потоков больше количества ядер (16, 128, 1024) ускорение не растёт: одновременно выполняются только 8 потоков, остальные ожидают своей очереди у планировщика, а эффективность падает примерно как 8/p. При 1024 потоках и N = 1000 части потоков вообще не достаётся строк, а время тратится на их создание и переключение контекста.

Ускорение растёт с увеличением размера матриц: на 8 потоках 2,75 при N = 250 и 6,02 при N = 1500. При N = 250 всё умножение занимает около 29 мс, и на 1024 потоках затраты на создание потоков превышают полезную работу (S = 0,68), а при N = 1500 накладные расходы малы по сравнению с вычислениями.

Основной сложностью оказалось честное сравнение версий: сначала последовательная версия использовала функцию matrix_multiply, в которой компилятор встраивал операции с комплексными числами, а функция потоков вызывала их из другого файла, из-за чего параллельная версия даже на одном потоке работала в 3,4 раза медленнее. После того как обе версии стали использовать одну функцию, ускорение на одном потоке стало равно 1. Корректность параллельного умножения проверена сравнением с последовательным, отсутствие гонок данных – с помощью ThreadSanitizer.